

Pull Request Workflow

Overview

This document defines the complete workflow for creating, reviewing, and merging Pull Requests in the greenfield_mp repository.

Workflow Summary

Feature Branch → Open PR → Automated Checks → Manual Review → Approve → Merge → Delete Branch

Step-by-Step Process

Step 1: Create a Feature Branch

```
# Start from develop (for features) or main (for hotfixes)
git checkout develop
git pull origin develop

# Create and switch to new branch
git checkout -b feature/your-feature-name
# or
git checkout -b bugfix/your-bugfix-name
# or
git checkout -b hotfix/your-hotfix-name
```

Step 2: Make Changes

```
# Make your changes
# Stage specific files
git add path/to/file.py

# Commit with descriptive message
git commit -m "Add feature X

- Implement Y functionality
- Update Z module
- Add tests for new feature

Closes #123"
```

Step 3: Push and Open PR

```
# Push your branch
git push -u origin feature/your-feature-name
```

```
# Open PR on GitHub targeting 'develop' (or 'main' for hotfixes)
```

Step 4: Automated Checks

Once PR is opened, the following checks run automatically: - **Linting**: Code style validation - **Unit Tests**: All unit tests must pass - **Integration Tests**: Database and API integration tests - **Security Scan**: Dependency and code security checks - **Build Verification**: Ensure code compiles/builds correctly

Step 5: Review Process

1. **Self-Review**: Reviewer should complete self-review checklist before submitting
2. **Automated Review**: All checks must pass
3. **Manual Review**: At least one team member must review and approve
4. **Address Feedback**: Make changes as requested and push updates

Step 6: Merge

After all checks pass and review is approved:

```
# Squash and merge (default for feature branches)
# In GitHub UI: click "Squash and merge"
# This creates a single clean commit with all changes
```

Step 7: Cleanup

```
# Delete local branch
git branch -d feature/your-feature-name

# Delete remote branch (optional - GitHub can do this automatically)
git push origin --delete feature/your-feature-name
```

PR Template

```
## Summary
[Brief description of changes]

## Related Issue
[Closes #issue_number] or [Related to #issue_number]

## Type of Change
- [ ] Bug fix
- [ ] New feature
- [ ] Breaking change
- [ ] Documentation update
- [ ] Refactor
- [ ] Test update

## Checklist
```

- [] Code follows project style guidelines
- [] Self-review completed
- [] Tests added or updated
- [] Documentation updated
- [] No breaking changes (or breaking changes documented)

Screenshots (if applicable)
[Add screenshots or GIFs]

Testing Performed
[Describe testing performed]

Deployment Notes
[Any special deployment instructions]

Review Checklist

For Reviewers

- [] Code compiles/runs without errors
- [] Follows PEP 8 and project conventions
- [] Meaningful variable and function names
- [] Proper error handling and logging
- [] No commented-out code or debug statements
- [] Tests added/modified as needed
- [] Documentation updated
- [] Security considerations addressed
- [] Performance implications considered
- [] Backward compatibility maintained

For PR Authors (Self-Review)

- [] I have reviewed my own code
- [] I have tested the changes locally
- [] I have updated relevant documentation
- [] I have added or updated tests
- [] I have checked for any breaking changes

Common Issues to Avoid

Commit Messages

- ❌ "Fixed stuff" / "Update" / "Changes"
- ✅ "Fix null pointer in employee validation"
- ✅ "Add SCD Type 2 refresh endpoint"

PR Descriptions

- ❌ Empty or vague descriptions
- ✅ Clear summary with context and testing notes

Code Changes

- ❌ Large unrelated changes in one PR
- ✅ Focused changes addressing a single issue or feature

Emergency Procedures

Hotfix Process

1. Create branch from `main`
2. Make critical fix
3. Open PR targeting `main`
4. Fast-track review (2 reviewers minimum)
5. Merge immediately after approval
6. Cherry-pick to `develop` if needed

Rollback Procedure

1. If merged PR causes issues, revert the commit
2. Create new PR for fix
3. Deploy fix immediately

Communication

- Use GitHub PR comments for discussions
- Use @mentions to notify team members
- Use labels to categorize PRs (bug, feature, documentation, etc.)
- Use milestones to track progress toward releases